Agenda

1. Types of LLD / OOD Interviews
2. How to approach different LLD Interviews
3. Design a Pen → AMAZON

Types of LLD / OOD Interviews

THEORETICAL
- → Old Tech companies / Mid Tier Startup
- → Mic / Amazon / Oracle / Services
- → Test knowledge
- → Theoretical ques^n about OOP / Design Patterns
- → 30-45 mins

DESIGN
- → Problem Solving Round → Single line
- → Case Study
- → NO CODE WORKING
- → MNCs
- → Microsoft / Amazon / Google / Walmart
- → 45 mins

MACHINE CODING
- → WORKING CODE
- → Extension of design
- → 2hr 30 mins
- → Top Startup
- → FK / Phonepe / Cred / Curefit / Scaler / Unacademy
- → Case Study

STEPS IN

Design and Machine Coding Rounds

DESIGNS (40 mins)

→ Single line problem statement
eg: "Design BookMyShow"

1. Gather Requirements
   ↳ clarify requirements
2. Use Case Diagram (~2-3%)
   (ask interviewer)
3. Class Diagram

→ 10 mins / 15

→ 5-8 mins

MACHINE CODING (2hr 30 mins)

→ Detailed documented problem statement

1. Clarifying Requirements → 5-10m
2. Class Diagram → 10-15mm
3. Schema Design → 8-10mm
4. Working Code

→ classes to rep entities
→ interfaces
→ design pattern

10-12 mins

(4) Schema Design
(ask interviewer)

5-8 mins

(5) Write on paper/board some class / interface

Remaining Time

30 Min

1 hr 30 min

20-30 min

Asks q^n about code / follow up q^n

In Scaler

API Design

→ Single line Problem Statement
→ Gather Req
→ Clarify Req → API Design (Use Case Diag)
→ Class Diagram
→ Schema Design
→ Code

```
POST    /parking lot
{
    address:
    name:
}
```

```
GET    /parking lot/{id}
{
    vehicleType:
}
```

DEEP DIVE ON EACH STEP

(+ Design a Pen)

① OVERVIEW

→ Expectations from interviewer

① Web API v/s CLI (How to take input)

② What kind of system

① Entity

② Interactive System

② Web App^n

→ What problem statement is system solving

Kind of Systems → Not Machine coding

- → Entity: Just rep classes to rep the entity
- → Interactive: No persistence (Games)
- → Web System: Persistence (BMS, Splitwise)

How to take input

① Via command line → NO FRAMEWORK

② REST APIs → FRAMEWORK

PROBLEM STATEMENT

We already know about system

① align yourself with interviewer's expectation

→ letting what you know.

We don't know about system already

① ask interviewer about the problem statement

② only entity design or implement

If implement ③ do we have to persist data

④ how will system take input

Design a Pen

Gathering Req

II
- → Suggest ideas with rationale, if possible
- → 5-8 core features of system

(3-4 core features)

→ Visualize when you get overview

0. Anything that can write is a Pen
1. Diff kind of Pens (Ball, Gel, Fountain)
2. Pens will have refil. Refil has ink
3. Fountain Pen directly has ink
4. Ink can be of diff colour
5. Every pen will have brand and name and price
6. For some pens we may be able to change refil. For other we might not
7. Refil has a Nib. For Fountain, pen has nib.
8. Nib can be of diff radius.
9. Ink can have diff features eg WATERPROOF, TRANSPARENT.

Clarifying Req

III

→ Behaviours

→ How diff features are going to work

III

→ Edge cases

→ Future modn

write()

refill()

→ Multiple pens might have same write algorith

→ some pens can be refilled, others cannot

→ Based on input, we have to create pen of the type

IV Use Case Diagram

V Class Diagram

→ ① Visualization → Entity, Game (Non Machine coding)

② finding nouns in requirements → Machine Coding

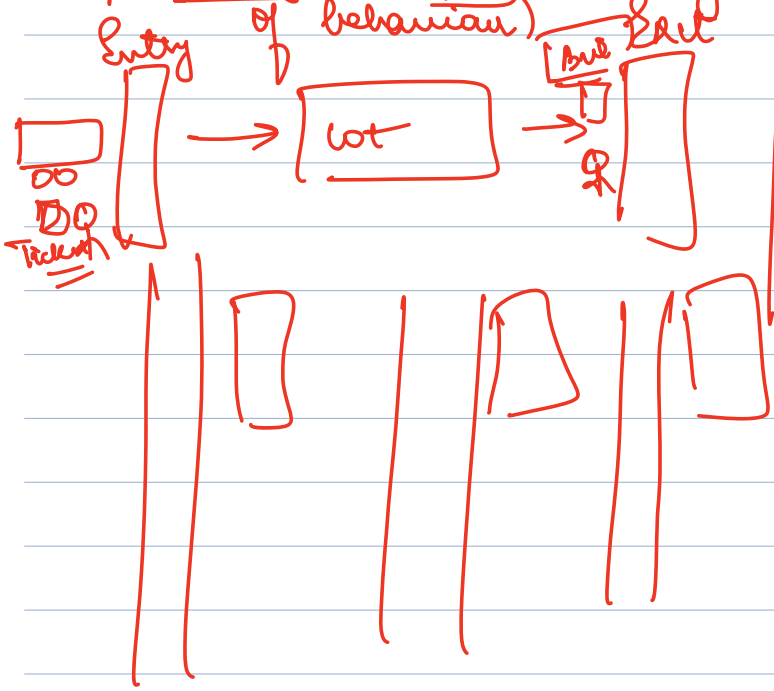
① Only expected to show classes for entities. (No controllers, services, helper) (going to be stored)

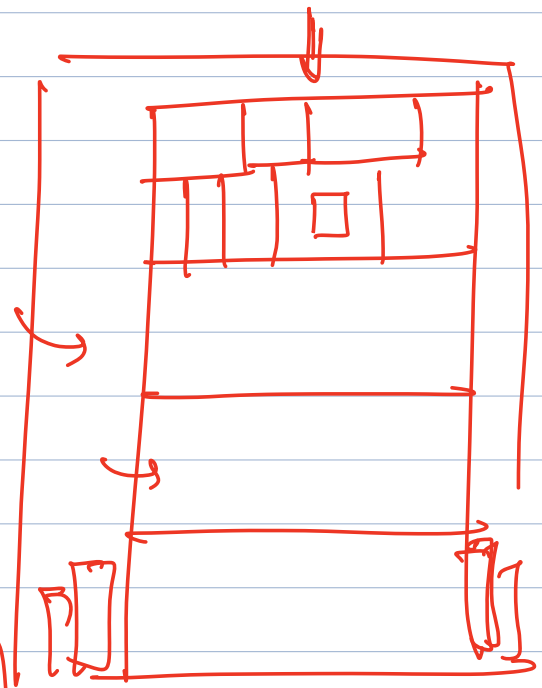
② Classes / interfaces for any design pattern

HOW TO VISUALIZE

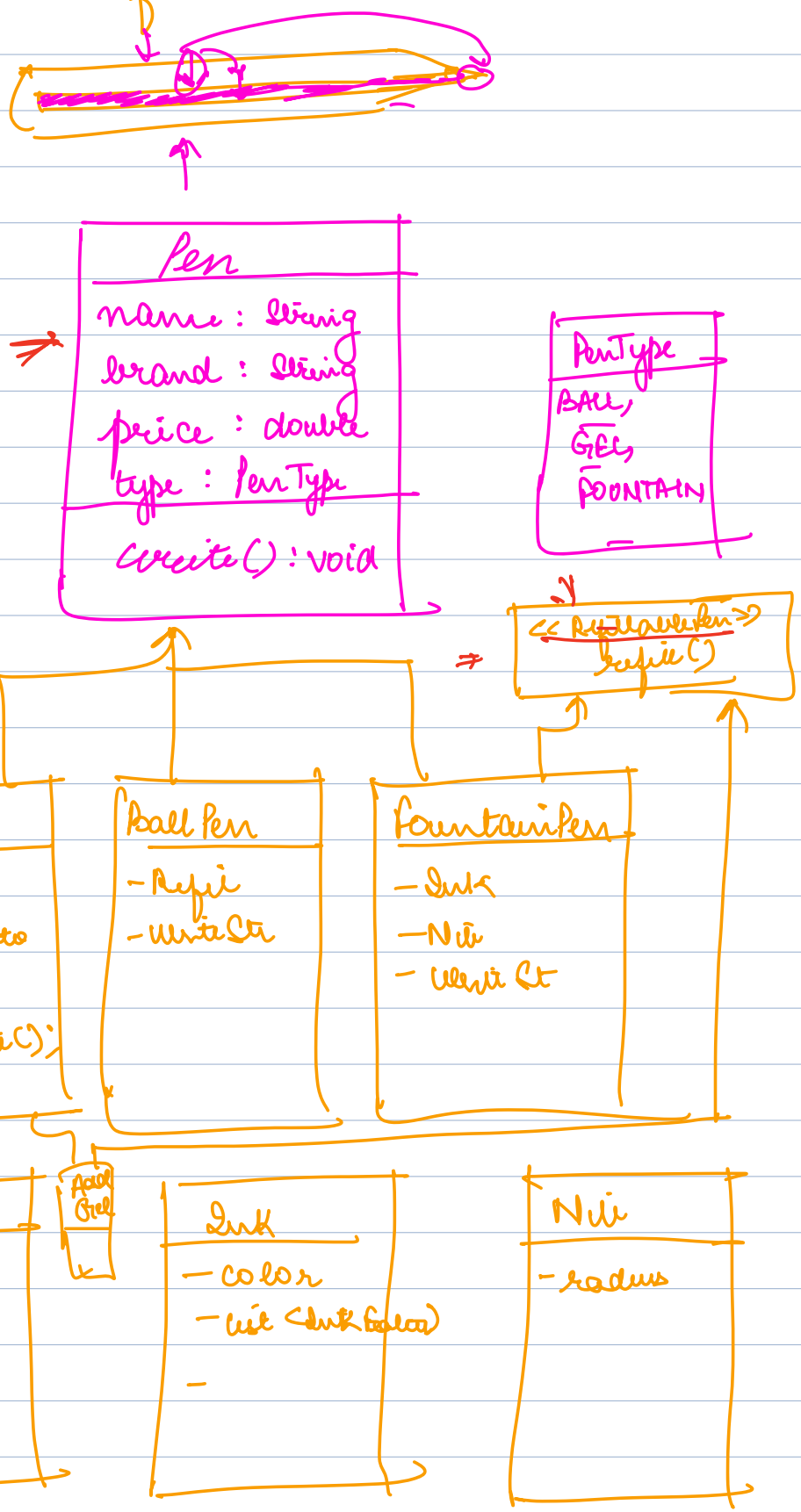
① Map a sketch for system
→ Go top to bottom

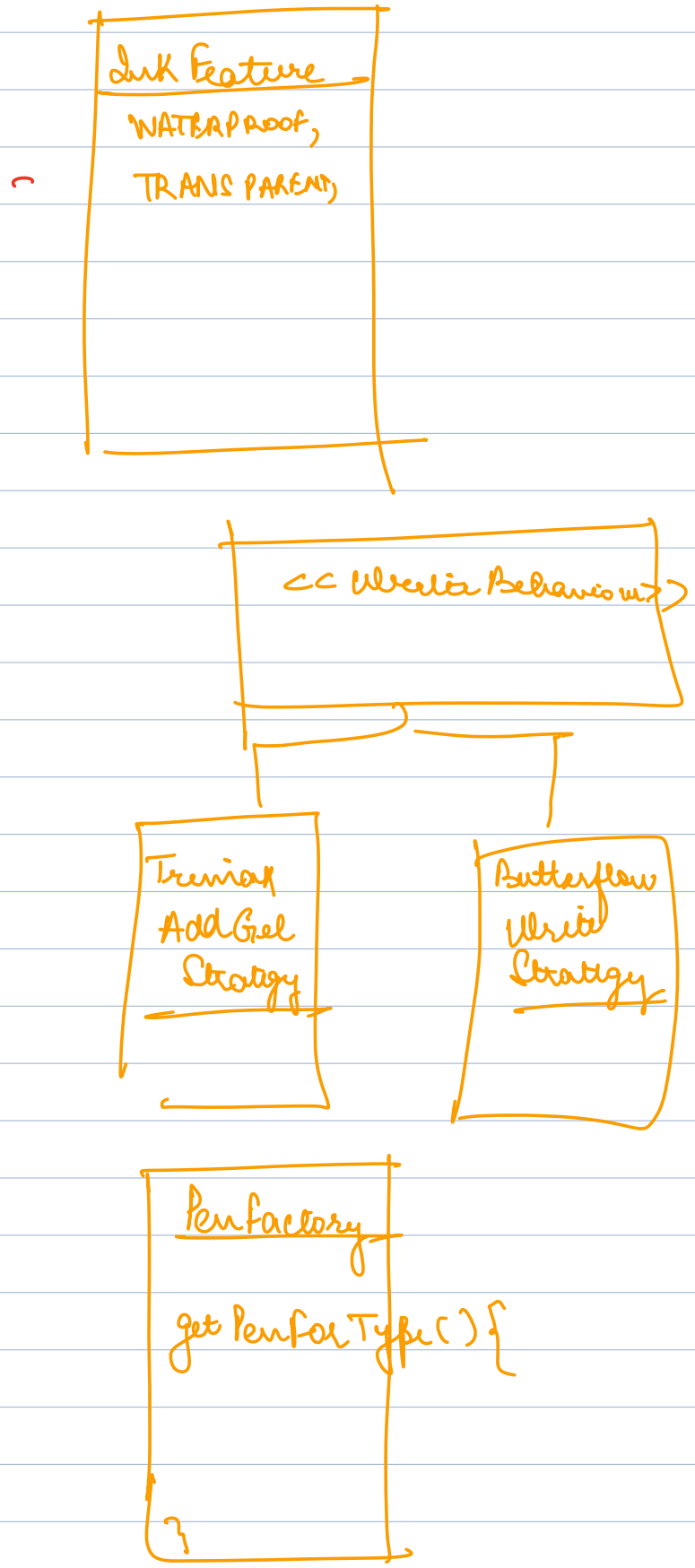
② User journeys (working of behavior)





Class Diagram of Pen




Ink Feature
WATERPROOF,
TRANSPARENT,
<< Writing Behaviour >>
Tremor
AddGel
Strategy
Butterflow
Write
Strategy
PenFactory
getPenForType() {
}

Next Class

- → How to Schema Design
- → How to Code (Machine Coding) ⇐ {PL, BMS, Spectrum, TTT}
- → Design of TTT